

Capítulo 5. Integración de conceptos: construyendo un programa completo

Caso de estudio: Sistema de Control de Parqueos

Table of contents

Introducción	2
Objetivos	2
Caso de estudio	2
Sistema de Control de Parqueos	2
Análisis del problema	2
Variables	3
Pseudocódigo	3
Diagrama: Sistema de control de parqueos	5
Paso 1	6
Paso 2	6
Paso 3. Entrada	7
Paso 4. Tarifa	7
Paso 5	7
Paso 6	7
Reflexión	8

Resumen	8
Paso 7. Mostrando los resultados	8
Ejemplo de ejecución	9
Reflexionemos	10
Paso 8. Analizando el programa	10
Paso 8. ¿Cómo podemos mejorarlo?	10
Lo que construiremos a continuación	10
Preparándonos para la siguiente clase	11

Introducción

Este capítulo integra los conceptos estudiados hasta el momento para resolver un problema completo en C++.

Objetivos

- Integrar variables, operadores y estructuras de control.
- Analizar un problema antes de programar.
- Diseñar una solución mediante pseudocódigo y Mermaid.

Caso de estudio

Sistema de Control de Parqueos

Análisis del problema

Entradas	Procesos	Salidas
Nombre	Determinar tarifa	Total
Placa	Calcular	Resumen
Tipo	Validar	Mensajes
Horas	Operaciones	Información

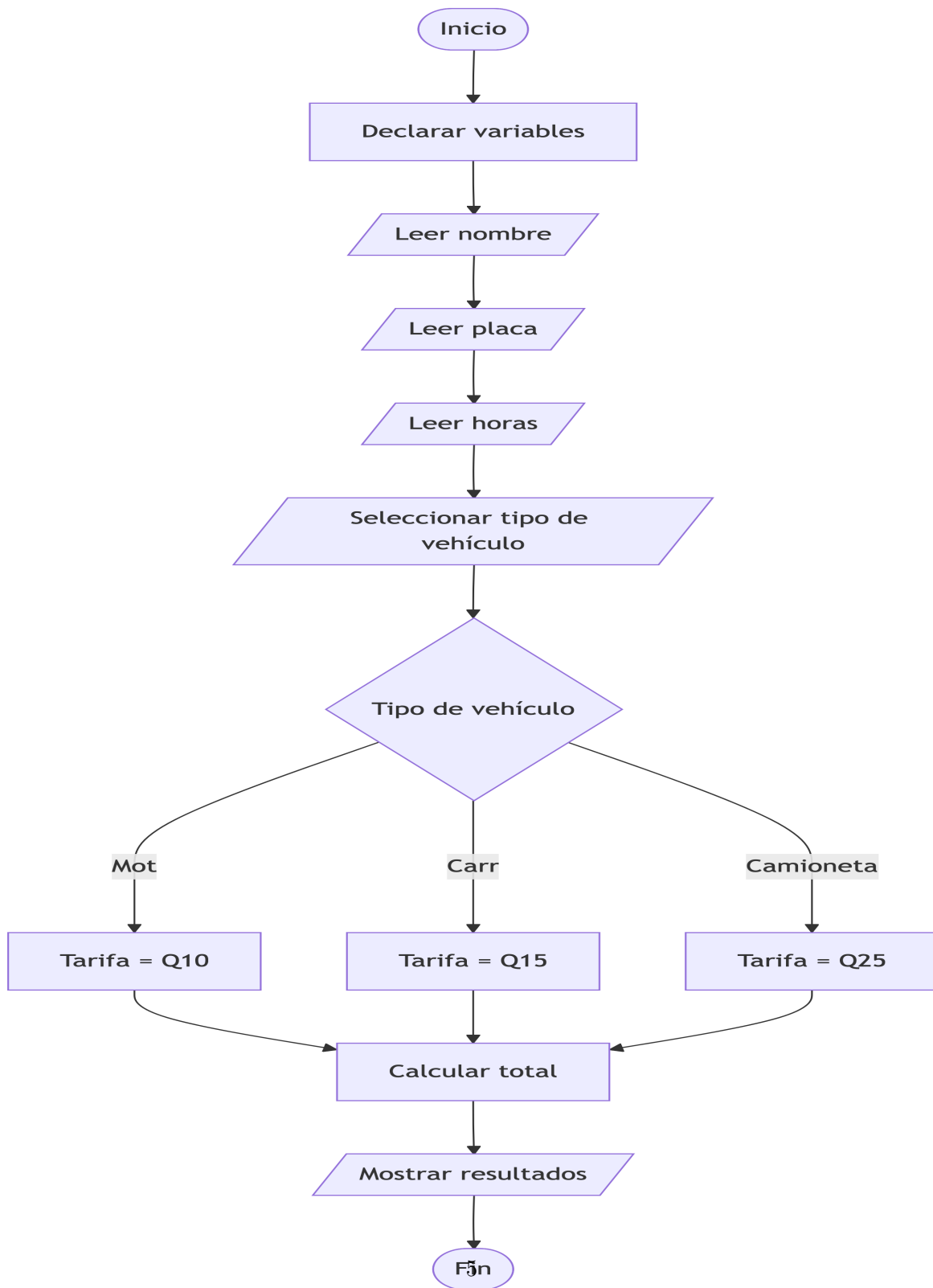
Variables

Variable	Tipo
nombreConductor	string
placa	string
tipoVehiculo	int
horas	int
tarifa	double
totalPagar	double

Pseudocódigo

```
Inicio
  Declarar variables
  Leer datos
  Determinar tarifa
  Calcular total
  Mostrar resultados
Fin
```


Diagrama: Sistema de control de parqueos



Paso 1

```
#include <iostream>
#include <string>
using namespace std;
```

Paso 2

```
//-----
// Enumeración de los tipos de vehículos
//-----

enum TipoVehiculo
{
    MOTO = 1,
    CARRO,
    CAMIONETA
};

//-----
// Declaración de variables
//-----

// En esta sección declaramos todas las variables que
// utilizaremos durante el programa, aunque algunas
// todavía no se empleen en los pasos siguientes.
// Esto mejora la organización y facilita la lectura
// del código.

string nombreConductor;
string placa;

int tipoVehiculo;
int horas;

double tarifa = 0.0;
double totalPagar = 0.0;
```

Paso 3. Entrada

```
cout<<"Nombre: ";
getline(cin,nombreConductor);
cout<<"Placa: ";
getline(cin,placa);

cout<<"Horas: ";
cin>>horas;

cout << "Tipo de vehículo" << endl;
cout << "1. Moto" << endl;
cout << "2. Carro" << endl;
cout << "3. Camioneta" << endl;
cout << "\nSeleccione una opción: ";
cin >> tipoVehiculo;
```

Paso 4. Tarifa

```
switch(tipoVehiculo){
case MOTO: tarifa=10; break;
case CARRO: tarifa=15; break;
case CAMIONETA: tarifa=25; break;
default: tarifa=0;
}
```

Paso 5

```
totalPagar=tarifa*horas;
```

Paso 6

```
cout<<"Total: Q"<<totalPagar<<endl;
```

Reflexión

- ¿Por qué `tarifa` es `double`?
- ¿Qué hace `switch`?

Resumen

Tema	Aplicado
Variables	
Operadores	
switch	
if	
while	
do...while	
for	

Paso 7. Mostrando los resultados

Una vez calculado el total a pagar, es momento de presentar la información al usuario de forma clara y ordenada.

```
//-----  
// Mostrar resultados  
//-----  
  
cout << "\n=====\\n";  
cout << "        SISTEMA DE CONTROL DE PARQUEOS\\n";  
cout << "=====\\n\\n";  
  
cout << "Conductor : " << nombreConductor << endl;  
cout << "Placa      : " << placa << endl;  
  
cout << "Tipo de vehículo" << endl;  
cout << "1. Moto" << endl;
```

```

cout << "2. Carro" << endl;
cout << "3. Camioneta" << endl;
cout << "\nSeleccione una opción: ";
cin >> tipoVehiculo;

switch(tipoVehiculo)
{
    case MOTO: cout << "Moto"; break;
    case CARRO: cout << "Carro"; break;
    case CAMIONETA: cout << "Camioneta"; break;
}

cout << endl;
cout << "Horas      : " << horas << endl;
cout << "Tarifa      : Q" << tarifa << endl;
cout << "-----" << endl;
cout << "TOTAL A PAGAR : Q" << totalPagar << endl;

```

Ejemplo de ejecución

Entrada

```

Nombre: Jorge García
Placa: P123ABC
Tipo de vehículo: 2
Horas: 3

```

Salida

```

=====
          SISTEMA DE CONTROL DE PARQUEOS
=====

Conductor : Jorge García
Placa      : P123ABC
Vehículo   : Carro
Horas      : 3
Tarifa     : Q15

-----
TOTAL A PAGAR : Q45

```

Reflexionemos

- ¿Por qué es importante presentar la información de forma ordenada?
- ¿Qué ventajas tiene mostrar el nombre del vehículo en lugar del número seleccionado?
- ¿Qué parte del programa realiza el cálculo y cuál solamente muestra información?

Paso 8. Analizando el programa

Concepto	¿Dónde aparece?
Variables	Declaración de variables
Operadores	<code>tarifa * horas</code>
<code>switch</code>	Selección de la tarifa
<code>cout</code>	Presentación de resultados

Hasta este punto ya tenemos un programa completamente funcional.

Paso 8. ¿Cómo podemos mejorarlo?

Nuestro programa funciona correctamente, pero como ocurre en el desarrollo de software real, siempre podemos incorporar nuevas características.

Cada mejora será la oportunidad perfecta para recordar una estructura de control ya estudiada.

Necesidad del programa	Concepto que utilizaremos
Validar las horas ingresadas	<code>if</code>
Clasificar el tiempo de permanencia	<code>if...else</code>
Aplicar descuentos	<code>if...else if</code>
Registrar varios vehículos	<code>while</code>
Mostrar un menú principal	<code>do...while</code>
Mostrar los espacios disponibles	<code>for</code>

Lo que construiremos a continuación

En la siguiente versión del programa incorporaremos:

1. Validación de datos.
2. Descuentos según el tiempo de permanencia.

3. Registro de varios vehículos.
4. Menú principal.
5. Visualización de espacios del parqueo.

Recuerda

Un buen programador no memoriza estructuras de control.
Las utiliza para resolver problemas reales.

Preparándonos para la siguiente clase

En la próxima sesión reorganizaremos este mismo programa utilizando:

- Funciones
- Procedimientos
- Parámetros

El objetivo será comprobar cómo un programa que ya funciona puede organizarse de una manera más profesional.